

Lab 2 Activity - Optimizing Sorts

Course: Analysis of Algorithms, Fall 2026

Instructor: Dr. Aaron Mininger

Due: Sept 24 at 11:59pm

Submit on Canvas: your modified sort algorithm (java file) and two powerpoint slides

Setup

Pull the most recent version of the course repository from Github:

<https://github.com/amininger-mu/CIS387F26Code>

All the code examples should be run from inside the **labs/lab2_sorting** directory. Compile everything using **javac *.java**

Code Organization

There are 5 sorting algorithms in the directory: insertion, selection, shell, merge, and quick.

Each one implements a standard **sort(int[] a)** interface as defined in **Sort.java**. That file also contains a number of helper methods for testing sorts.

Each sort can be run via its main function, for example: **java Selection.java 100**

You can also run a test suite for a particular algorithm via: **java TestSort insertion** (you can give any of the 5 sort names). Use this program to make sure your modified sort works.

You can compare two algorithms using **java CompareSorts <alg1> <alg2> <length> <nTrials>**

- **alg1** and **alg2** are the names of two sorting algorithms
- **length** is an int which is the size of arrays to create when testing
- **nTrials** is an int which says how many tests to run for each algorithm

The program will n trials for each algorithm, each time sorting a random array of the given length. It adds up the total time for each algorithm, and prints the results.

For example: **java CompareSorts merge quick 10000 10** will merge sort a random array of 10000 integers a total of 10 times. It will do the same for quick sort and compare the total times.

Lab Requirements

You will be assigned a particular optimization to make to one of the sorting algorithms. Make a copy of that sort's java file (you want the original to compare against) and then make the change. You may consult any resource (book, website, AI) you want, just make sure you can explain in your own words why and how your optimization works.

Once you have created a modified version of the sort, you should run it through **TestSort** to make sure it works:

- Add your sort as a new option in the function **parseSortAlgorithm** in **Sort.java** (with new name)
- Run **java TestSort <name>** to test your modified sort.

Once you are sure your change is working, run some tests to compare your modification to the original sort using **CompareSorts**:

- You want a large enough number of trials to have some confidence in the results (at least 10)
- You want to choose as large a length as possible that still completes in a reasonable time

Run with a few different combinations of length and trials until you gain some confidence in the performance difference your change made (how much faster/slower is it compared to the original?). Record your results.

Presentation

You should be ready to discuss your results in class on Friday. To help you, prepare 2 powerpoint slides (and send them to me):

1. Explain what optimization you made to a sort (and the theory behind why it should help)
2. Summarize your experimental results (how did this change affect the performance)

Plan on a brief, informal presentation of around 1-2 minutes.